Council Verdict: A.U.R.A. Feasibility

Where the Council Agrees

- **The scope is the single biggest risk, not the ideas.** Every individual piece (accessibility automation, quantized local LLMs, vector memory, biometric gating) is well-trodden ground. Building *all of them, integrated, working reliably* in one academic year, starting from zero, is the actual risk — not whether any one piece is technically possible.
- **CPU-only multi-hop LLM inference is the load-bearing assumption that will break first.** Brain → Manager → Sub-agent → (maybe) Vision fallback means a single user command could trigger 3–4 sequential LLM calls before anything happens on screen. On a 7–8B Q4 model on CPU, that's realistically several seconds *per hop* — a "close this window" command could take 15–30+ seconds end to end. This needs to be measured early, not assumed away.
- **The security story is stronger in framing than in current specification.** "Zero-trust," "AES-256," "hardware biometric gate" all sound rigorous, but key details that determine whether they're real or decorative aren't in the doc (key storage/derivation, Pi↔laptop channel authentication, whether sanitizing HTML actually blocks plain-text prompt injection).

Where the Council Clashes

- **Cut scope vs. keep scope.** The Executor and Contrarian want the Pi companion and vision fallback deferred or dropped as stretch goals, arguing they're the most hardware/compute-heavy, least core-to-the-thesis pieces. The Expansionist disagrees — the Pi + biometric HITL is arguably the most *distinctive* part of the project (most GUI-agent projects don't have a physical hardware trust anchor), and cutting it removes the thing that would make this stand out in a demo or a paper. Reasonable people can land either way depending on whether the goal is "ship something working" or "have the most interesting FYP."
- **Is "air-gapped, all-local" actually the right north star?** First Principles pushes back hardest here: the privacy/local-first pitch is compelling, but it's also what's forcing the

hardest constraint (CPU-only 7-8B inference) onto what should be a snappy, low-latency automation tool. The Contrarian and Executor are more sympathetic to keeping it as-is since it *is* the thesis and the novelty claim; without it, this is "yet another GUI agent."

Blind Spots the Council Caught

- **Accessibility-tree coverage was never stress-tested against real apps.** pywinauto's UIA backend degrades badly on Electron apps (VS Code, Slack, Discord, most modern chat/dev tools) and many Win32 legacy apps — this is exactly the class of app a CS student would actually demo with. If the vision fallback ends up firing 40-50% of the time instead of being a rare fallback, the "efficient, non-intrusive automation" pitch quietly becomes "a slow vision-based clicker with extra steps," and that changes the whole performance budget.
- **The Pi is a new attack surface, not just a security feature.** Nobody in the initial round flagged this, but it came up in review: whatever protocol the Raspberry Pi uses to tell the laptop "fingerprint approved, go ahead" is now part of the trust boundary. If that channel isn't itself authenticated/encrypted, an attacker who can reach it (same LAN, spoofed device, etc.) could approve high-risk actions without ever touching the fingerprint sensor — quietly undermining the entire "hardware, not just software, gate" claim.
- **Novelty vs. prior art wasn't addressed.** 50 sources were reviewed, but the doc doesn't stake out what's actually new here versus existing computer-use/GUI agents (both commercial and open-source) that already do accessibility-first automation with vision fallback. A FYP committee will ask this directly; it's worth having a one-line answer ready ("what does A.U.R.A. do that [X] doesn't") rather than building it live during defense.

The Recommendation

Practically possible — but only if you treat this as a **staged system with a real MVP**, not five phases built in sequence to completion. Concretely:

1. **Descope the Pi/biometric hardware to Phase 5 "if time allows," not a core deliverable.** Prototype the biometric gate in software first (a confirm dialog + simulated

approval flow) and swap in real hardware only once the core loop works. This is reversible later; a broken core loop with fancy hardware attached is not a good demo.

2. **Benchmark the two-tier LLM latency in week one, before building anything else.** Run BitNet 2B and your chosen 7–8B quant on your actual dev laptop, on a realistic multi-hop prompt chain, and measure real wall-clock time. This number should drive every other design decision (how many hops per command, whether Manager decomposition is even needed for simple commands, whether you need a "fast path" that skips the full agent hierarchy for trivial actions).
3. **Tighten the security claims to match what you'll actually implement.** Specify key storage/derivation for the AES-256 layer and the authentication story for the Pi↔laptop channel now, on paper, even before building — reviewers will probe exactly these gaps, and "we assumed the OS keychain" is a fine answer if it's a decision, not an oversight.
4. **Write one paragraph on novelty vs. existing GUI/computer-use agents** and keep it visible in the doc — this pre-empts the most predictable committee question.

The One Thing to Do First

Before writing any agent-orchestration code: install llama.cpp, pull BitNet 1.58b 2B and a 7–8B Q4 model, and time a 3-hop chained inference (Brain → Manager → Sub-agent) on your actual target hardware for a trivial command. That single number tells you whether the architecture as designed is usable at all, or needs restructuring — and it's a half-day of work that de-risks everything else in the plan.